

Combined Hotspots and Materials Product

Introduction

The following sections demonstrate that the HEATWISE Combined Hotspots and Materials Software follows the OGC Best Practice for Earth Observation Application Package.

The order of demonstration is deliberately different from that given in the best practice document. This is done to make the demonstration easy to follow and group requirements that can be shown together. An effort is made to keep the requirements in order as much as possible to make it easy to find a specific requirement.

The demonstration proceeds in the following steps:

- Execute the software as docker container and as CWL document (the EOAP)
- Show that STAC inputs and outputs are handled by the application as required
- Show that the CWL document is structured as required
- Show that STAC inputs and outputs are declared in the CWL as required

After each step, the requirements shown are listed.

@sec-requirements contains an overview of all requirements in order and how they are demonstrated in this document. It furthermore shows all utility scripts used in this demonstration. For brevity, the content of utility scripts only shown in the requirements overview and not in each demonstration.

Preparation

Activate the pixi environment to make validation tools and scripts available.

```
eval "$(pixi shell-hook --change-ps1 false)"
```

Set environment variables to software, cwl documents, inputs and outputs.

```
set -o pipefail
set -x
SOFTWARE_ROOT="$PIXI_PROJECT_ROOT"/software/lstm-wp3-products/products/ua
CWL_PATH="$PIXI_PROJECT_ROOT"/application-packages/heatwise-combined-hotspots-
materials.cwl
INPUT_PATH="$PIXI_PROJECT_ROOT"/inputs/combined_hotspots_materials

DOCKER_IMAGE_ID=ghcr.io/esa-heatwise/lstm-wp3-products-ua:latest
OUTPUT_PATH="$PIXI_PROJECT_ROOT"/outputs/combined_hotspots_materials/
```

```
ls -ltrAh --color=never "$SOFTWARE_ROOT"
set +x
```

```
+ SOFTWARE_ROOT=/home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/software/lstm-wp3-products/products/ua
+ CWL_PATH=/home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/application-packages/heatwise-combined-hotspots-materials.cwl
+ INPUT_PATH=/home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/inputs/combined_hotspots_materials
+ DOCKER_IMAGE_ID=ghcr.io/esa-heatwise/lstm-wp3-products-ua:latest
+ OUTPUT_PATH=/home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/outputs/combined_hotspots_materials/
+ ls --color=auto -ltrAh --color=never /home/hannes/projects/heatwise/heatwise-
interoperability-demonstration/software/lstm-wp3-products/products/ua
total 184K
-rw-rw-r-- 1 hannes hannes 1.1K Sep 18 11:17 README.md
-rwxrwxr-x 1 hannes hannes 257 Sep 18 11:17 build.sh
drwxrwxr-x 2 hannes hannes 4.0K Sep 18 11:17 app
drwxrwxr-x 2 hannes hannes 4.0K Sep 18 11:17 data
-rw-rw-r-- 1 hannes hannes 126 Sep 18 11:17 environment.yml
drwxrwxr-x 5 hannes hannes 4.0K Sep 18 11:17 inputs
-rw-rw-r-- 1 hannes hannes 7.2K Sep 18 11:17 ua_attrs.ipynb
-rw-rw-r-- 1 hannes hannes 154 Sep 18 11:17 run.sh
-rw-rw-r-- 1 hannes hannes 145K Sep 18 14:29 ua_attrs.pdf
+ set +x
```

Running the software

The first set of requirements is concerned with how the software is executed. It shall be

- Runnable as a non-interactive script
- Bundled in (Docker) container
- Executable as a CWL Workflow containing at least one CWL CommandLineTool

Running as a docker container

We can run the (dockerized) software directly, without using the CWL machinery with `docker run`:

```
mkdir -p "$OUTPUT_PATH"
docker run --rm --user "$(id -u):$(id -g)" --workdir=/home/mambauser/wd
-v $SOFTWARE_ROOT/data:/home/mambauser/data -v $SOFTWARE_ROOT/inputs:/home/
mambauser/wd/inputs -v "$OUTPUT_PATH":/home/mambauser/wd $DOCKER_IMAGE_ID ||
```

```
{ FAIL_REQ_1=1; FAIL_REQ_2=1; }  
ls -ltrAh --color=never $OUTPUT_PATH
```

```
INFO:pyogrio._io:Created 769 records  
CWD /home/mambauser/wd  
total 16K  
drwxr-xr-x 2 root root 4.0K Sep 18 11:19 inputs  
drwxrwxr-x 4 hannes hannes 4.0K Sep 18 14:15 cwlruns  
drwxr-xr-x 2 hannes hannes 4.0K Sep 18 14:30 datasets_saved  
-rw-r--r-- 1 hannes hannes 367 Sep 18 14:30 catalog.json
```

The image should not only be available locally, but also in a container registry.

```
check-container-registry.sh $DOCKER_IMAGE_ID || FAIL_REQ_3=1
```

```
SUCCESS: Image ghcr.io/esa-heatwise/lstm-wp3-products-ua:latest found on  
registry
```

Requirements Validated

req/app/cmd-line (Requirement 1)

The Application SHALL be a non-interactive executable as a command-line application.

req/app/container (Requirement 2)

The environment, libraries, binaries, executable and configuration files necessary to execute the Application SHALL be bundled in a container image.

req/app/registry (Requirement 3)

The Application container image SHALL be accessible in a container registry.

Running as an EOAP

First, we validate that that the CWL document is valid.

```
cwltool --disable-color --validate "$CWL_PATH" | cat
```

```
INFO /home/hannes/projects/heatwise/heatwise-interoperability-  
demonstration/.pixi/envs/dev/bin/cwltool 3.2.20260413085819
```

```

INFO      Resolved      '/home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/application-packages/heatwise-combined-hotspots-materials.cwl'
to          'file:///home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/application-packages/heatwise-combined-hotspots-materials.cwl'
WARNING:root:File contains $graph of multiple objects and no default process
(#main). Validating all objects:
file:///home/hannes/projects/heatwise/heatwise-interoperability-demonstration/
application-packages/heatwise-combined-hotspots-
materials.cwl#heatwise_combined is valid CWL.
file:///home/hannes/projects/heatwise/heatwise-interoperability-demonstration/
application-packages/heatwise-combined-hotspots-materials.cwl#xce_script      is
valid CWL.
/home/hannes/projects/heatwise/heatwise-interoperability-demonstration/
application-packages/heatwise-combined-hotspots-materials.cwl is valid CWL.

```

The CWL should contain a `Workflow` and at least one `CommandLineTool` object. We inspect the classes in the CWL graph to make sure.

```
show-cwl-graph-classes.sh "$CWL_PATH"
```

```

class: Workflow
id: heatwise_combined
---
class: CommandLineTool
id: xce_script

```

The EOAP can be executed from its CWL description with `cwltool`.

```

cwltool \
  --disable-color \
  --outdir "$OUTPUT_PATH/cwlruns" \
  "$CWL_PATH"#heatwise_combined \
  \
  --urban_atlas $INPUT_PATH/urban_atlas \
  --hcspots $INPUT_PATH/hcspots \
  --material_labels $INPUT_PATH/material_labels \
  || FAIL_REQ_7=1

```

```

INFO      /home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/.pixi/envs/dev/bin/cwltool 3.2.20260413085819
INFO      Resolved      '/home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/application-packages/

```

```

heatwise-combined-hotspots-materials.cwl#heatwise_combined' to 'file:///home/
hannes/projects/heatwise/heatwise-interopability-demonstration/application-
packages/heatwise-combined-hotspots-materials.cwl#heatwise_combined'
INFO [workflow ] start
INFO [workflow ] starting step run_script
INFO [step run_script] start
INFO [job run_script] /tmp/6ljspay0$ docker \
    run \
    -i \
    --mount=type=bind,source=/tmp/6ljspay0,target=/OZKhZt \
    --mount=type=bind,source=/tmp/2e6hzouc,target=/tmp \
    --mount=type=bind,source=/
home/hannes/projects/heatwise/heatwise-interopability-demonstration/inputs/
combined_hotspots_materials/hcspots,target=/var/lib/cwl/stg33adb7a2-b86e-40f4-
a729-23d90810c79c/hcspots,readonly \
    --mount=type=bind,source=/home/hannes/projects/heatwise/
heatwise-interopability-demonstration/inputs/combined_hotspots_materials/
material_labels,target=/var/lib/cwl/stgc759738b-afbf-41d0-83c1-910a7d32bd48/
material_labels,readonly \
    --mount=type=bind,source=/home/hannes/projects/heatwise/
heatwise-interopability-demonstration/inputs/combined_hotspots_materials/
urban_atlas,target=/var/lib/cwl/stg191f50ad-c76c-42ef-af96-bb9010b46eb8/
urban_atlas,readonly \
    --workdir=/OZKhZt \
    --read-only=true \
    --user=1000:1000 \
    --rm \
    --cidfile=/tmp/tfdsdxoq/20260918143014-670289.cid \
    --env=TMPDIR=/tmp \
    --env=HOME=/OZKhZt \
    ghcr.io/esa-heatwise/lstm-wp3-products-ua:latest \
    /usr/local/bin/_entrypoint.sh \
    python \
    /home/mambauser/execute.py \
    --batch \
    --eoap \
    --asset-id-hcspots \
    hcspots \
    --asset-id-mat \
    mat \
    --asset-id-ua \
    ua \
    --band-hotspot \

```

```

2 \
--hcspots \
/var/lib/cwl/stg33adb7a2-b86e-40f4-a729-23d90810c79c/hcspots \
--material-labels \
/var/lib/cwl/stgc759738b-afbf-41d0-83c1-910a7d32bd48/material_labels \
--material-legend \
asphalt,concrete,terracotta,vegetation,metal \
--material-ndv \
0 \
--savename \
hw_combined_sepolia.gpkg \
--urban-atlas \
/var/lib/cwl/stg191f50ad-c76c-42ef-af96-bb9010b46eb8/urban_atlas
INFO:pyogrio._io:Created 769 records
CWD /OZKhZt
INFO [job run_script] Max memory used: 5677MiB
INFO [job run_script] completed success
INFO [step run_script] completed success
INFO [workflow ] completed success
{
  "stac_catalog": {
    "location": "file:///home/hannes/projects/heatwise/
heatwise-interoperability-demonstration/outputs/combined_hotspots_materials/
cwlruns/6ljspay0",
    "basename": "6ljspay0",
    "class": "Directory",
    "listing": [
      {
        "class": "File",
        "location": "file:///
home/hannes/projects/heatwise/heatwise-interoperability-demonstration/outputs/
combined_hotspots_materials/cwlruns/6ljspay0/catalog.json",
        "basename": "catalog.json",
        "size": 367,
        "checksum": "sha1$04d3a7537257da5e802e0875ee13fe2a21bdc435",
        "path": "/home/hannes/projects/heatwise/
heatwise-interoperability-demonstration/outputs/combined_hotspots_materials/
cwlruns/6ljspay0/catalog.json"
      },
      {
        "class": "Directory",
        "location": "file:///
home/hannes/projects/heatwise/heatwise-interoperability-demonstration/outputs/

```

```

combined_hotspots_materials/cwlrns/6ljspay0/datasets_saved",
    "basename": "datasets_saved",
    "listing": [
        {
            "class": "File",
            "location": "file:///home/hannes/projects/heatwise/
heatwise-interoperability-demonstration/outputs/combined_hotspots_materials/
cwlrns/6ljspay0/datasets_saved/urban_atlas_with_hotspots.json",
            "basename": "urban_atlas_with_hotspots.json",
            "size": 1654956,
            "checksum": "sha1$17a45ad30d1810e9afc007392784e642592b3e2e",
            "path": "/home/hannes/projects/heatwise/
heatwise-interoperability-demonstration/outputs/combined_hotspots_materials/
cwlrns/6ljspay0/datasets_saved/urban_atlas_with_hotspots.json"
        },
        {
            "class": "File",
            "location": "file:///home/hannes/projects/heatwise/
heatwise-interoperability-demonstration/outputs/combined_hotspots_materials/
cwlrns/6ljspay0/datasets_saved/hw_combined_sepolia.gpkg",
            "basename": "hw_combined_sepolia.gpkg",
            "size": 839680,
            "checksum": "sha1$79f3d479b1d1ed7404e007ee70e4a5c2b7e43a30",
            "path": "/home/hannes/projects/heatwise/
heatwise-interoperability-demonstration/outputs/combined_hotspots_materials/
cwlrns/6ljspay0/datasets_saved/hw_combined_sepolia.gpkg"
        }
    ],
    "path": "/home/hannes/projects/heatwise/
heatwise-interoperability-demonstration/outputs/combined_hotspots_materials/
cwlrns/6ljspay0/datasets_saved"
}
],
    "path": "/home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/outputs/combined_hotspots_materials/cwlrns/6ljspay0"
}
}INFO Final process status is success

```

Cwltool creates a new output directory with a random name when it is run. We save the last run in a variable to access it later.

```

CWLRUN_PATH=$(find-newest-directory-entry.sh "$OUTPUT_PATH/cwlrns")
echo "Path to last cwl run: $CWLRUN_PATH"

```

```
Path to last cwl run: /home/hannes/projects/heatwise/heatwise-interoperability-
demonstration/outputs/combined_hotspots_materials/cwlruns/6ljspay0
```

Requirements Validated

req/app-pck/cwl (Requirement 7)

The Application Package SHALL be a valid CWL document with a “Workflow” class and one or more “CommandLineTool” classes.

EOAP Output

```
tree -n $CWLRUN_PATH || :
```

```
/home/hannes/projects/heatwise/heatwise-interoperability-demonstration/
outputs/combined_hotspots_materials/cwlruns/6ljspay0
├─ catalog.json
└─ datasets_saved
   └─ hw_combined_sepolia.gpkg
      └─ urban_atlas_with_hotspots.json
```

2 directories, 3 files

```
python -c "import geopandas as gpd; import
matplotlib.pyplot as plt; gpd.read_file(\"$CWLRUN_PATH/datasets_saved/
hw_combined_sepolia.gpkg\").plot(); plt.savefig('hw_combined_sepolia.png',
dpi=300, bbox_inches='tight')" || :
```

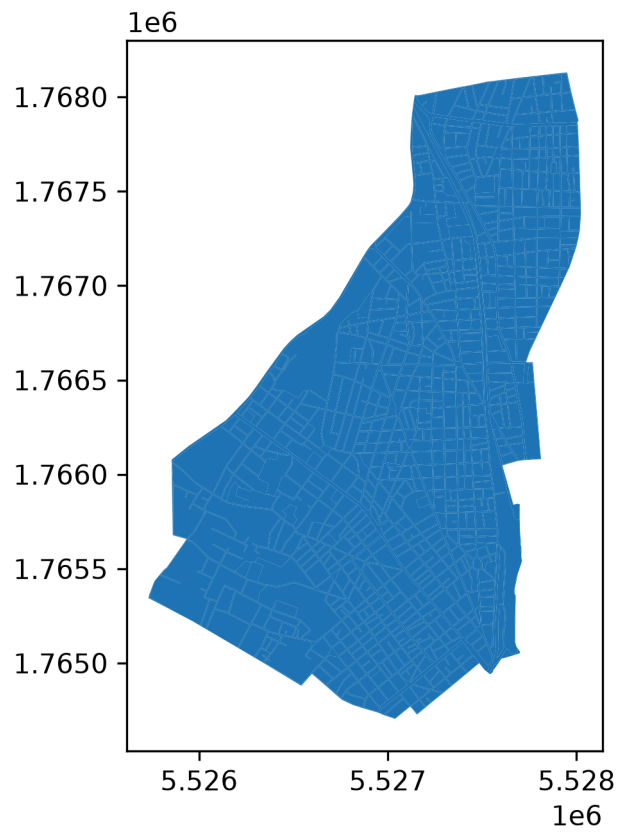


Figure 1: Combined Hotspots Materials output

STAC input / output

Inputs

The EO Products of the input are provided as directories with a `catalog.json` describing the products each:

```
ls -lAhX --color=never $INPUT_PATH/*
```

```
/home/hannes/projects/heatwise/heatwise-interoperability-demonstration/inputs/
combined_hotspots_materials/hcspots:
total 16K
-rw-rw-r-- 1 hannes hannes 338 Sep 18 11:17 catalog.json
-rw-rw-r-- 1 hannes hannes 505 Sep 18 11:17 hcspots_item.json
-rw-rw-r-- 1 hannes hannes 7.7K Sep 18 11:17 Sepolia_Thermopolis_LST-
N_Clusters.tif

/home/hannes/projects/heatwise/heatwise-interoperability-demonstration/inputs/
```

```
combined_hotspots_materials/material_labels:
total 12K
-rw-rw-r-- 1 hannes hannes 326 Sep 18 11:17 catalog.json
-rw-rw-r-- 1 hannes hannes 592 Sep 18 11:17 mat_item.json
-rw-rw-r-- 1 hannes hannes 2.4K Sep 18 11:17 Sepolia_material_label_dominant.tif

/home/hannes/projects/heatwise/heatwise-interopability-demonstration/inputs/
combined_hotspots_materials/urban_atlas:
total 828K
-rw-rw-r-- 1 hannes hannes 820K Sep 18 11:17 UA2021_Sepolia.gpkg
-rw-rw-r-- 1 hannes hannes 323 Sep 18 11:17 catalog.json
-rw-rw-r-- 1 hannes hannes 575 Sep 18 11:17 ua_item.json
```

These same inputs are classed as `Directory` in the workflow:

```
extract-directory-inputs-from.sh Workflow "$CWL_PATH" || FAIL_REQ_4=1
```

```
Running
inputs_with_directory_type:
- label: hcspots
  doc: hcspots
  type: Directory
  default:
    class: Directory
    location: null
- label: material_labels
  doc: material_labels
  type: Directory
  default:
    class: Directory
    location: null
- label: urban_atlas
  doc: urban_atlas
  type: Directory
  default:
    class: Directory
    location: null
```

Outputs

The output contains a single `catalog.json` and item, describing the output. Using the recursive validation, both the catalog and item are confirmed to be valid STAC.

```
(stac-validator validate --recursive $CWLRUN_PATH/catalog.json | cat) ||  
FAIL_REQ_5=1
```

Thanks for using STAC version 1.1.0!

```
[  
  {  
    "version": "1.1.0",  
    "path": "/home/hannes/projects/heatwise/  
heatwise-interopability-demonstration/outputs/combined_hotspots_materials/  
cwlruns/6ljspay0/catalog.json",  
    "schema": [  
      "https://schemas.stacspec.org/v1.1.0/catalog-spec/json-schema/  
catalog.json"  
    ],  
    "valid_stac": true,  
    "asset_type": "CATALOG",  
    "validation_method": "recursive",  
    "validator_engine": "jsonschema"  
  },  
  {  
    "version": "1.1.0",  
    "path": "/home/hannes/projects/heatwise/  
heatwise-interopability-demonstration/outputs/combined_hotspots_materials/  
cwlruns/6ljspay0/./datasets_saved/urban_atlas_with_hotspots.json",  
    "schema": [  
      "https://schemas.stacspec.org/v1.1.0/item-spec/json-schema/item.json"  
    ],  
    "valid_stac": true,  
    "asset_type": "ITEM",  
    "validation_method": "recursive",  
    "validator_engine": "jsonschema"  
  }  
]
```

```
-- Recursive Validation Summary:  
Stac objects passed: 2/2 (100.0%)
```

```
Breakdown by type:  
Catalog: 1/1 (100.0%)  
Item: 1/1 (100.0%)
```

```
Validation completed in 6.11s
```

The EOAP best practice mandates that the item contains at least `geometry`, `bbox` and `datetime` keys.

```
extract-output-keys-from-stac-item.sh          $CWLRUN_PATH/datasets_saved/  
urban_atlas_with_hotspots.json || FAIL_REQ_6=1
```

```
geometry:  
  type: MultiPolygon  
  coordinates: '[...truncated...]'  
bbox:  
  - 23.69854677617415  
  - 37.9856945053099  
  - 23.728626346125544  
  - 38.014987032184855  
properties:  
  datetime: '2026-09-18T12:30:18.089146Z'
```

```
Required keys present: true
```

Requirements Validated

req/app/stac-input (Requirement 4)

An Application input argument that requires staged EO product files SHALL be defined as an argument that points to a folder where a STAC Catalog, named `catalog.json`, contains a list of one or more STAC Items and associated STAC Assets referencing the files.

req/app/stac-out (Requirement 5)

An Application that creates EO product files to be stage-out SHALL generate a valid STAC Catalog, named `catalog.json`, and include the STAC Item(s) and corresponding STAC Assets pointing to the results of the processing.

rec/app/stac-out-metadata (Requirement 6)

The STAC Catalog created by the Application SHALL include metadata elements for each STAC Item with at least their spatial (geometry, box) and temporal (datetime) properties.

CWL

The fields `id`, `baseCommand`, `inputs`, `requirements` should be present in the `CommandLineTool`. One of the requirements should be `DockerRequirement`.

```
check-commandlinetool-fields.sh "$CWL_PATH" || FAIL_REQ_8=1
```

```
id: xce_script
baseCommand:
  - /usr/local/bin/_entrypoint.sh
  - python
  - /home/mambauser/execute.py
requirements:
  DockerRequirement:
    dockerPull: ghcr.io/esa-heatwise/lstm-wp3-products-ua:latest
Required keys present: true
```

The fields `id`, `label`, and `doc` should be present in the `Workflow`.

```
check-workflow-fields.sh "$CWL_PATH" || FAIL_REQ_10=1
```

```
id: heatwise_combined
label: xcengine notebook
doc: xcengine notebook
Required keys present: true
```

Each input of the `Workflow` should have fields `id`, `label`, and `doc`

```
check-workflow-input-fields.sh $CWL_PATH || FAIL_REQ_11=1
```

```
inputs:
  - id: asset_id_hcspots
    label: asset_id_hcspots
    doc: asset_id_hcspots
  - id: asset_id_mat
    label: asset_id_mat
    doc: asset_id_mat
  - id: asset_id_ua
    label: asset_id_ua
    doc: asset_id_ua
  - id: band_hotspot
    label: band_hotspot
```

```

    doc: band_hotspot
  - id: hcspots
    label: hcspots
    doc: hcspots
  - id: material_labels
    label: material_labels
    doc: material_labels
  - id: material_legend
    label: material_legend
    doc: material_legend
  - id: material_ndv
    label: material_ndv
    doc: material_ndv
  - id: savename
    label: savename
    doc: savename
  - id: urban_atlas
    label: urban_atlas
    doc: urban_atlas

```

The CWL document should contain a **version**:

```
check-version.sh $CWL_PATH || FAIL_REQ_9=1
```

```
s:version: 1.0.0
```

Requirements Validated

req/app-pck/clt (Requirement 8)

The Application Package CWL CommandLineTool classes SHALL contain the following elements: — Identifier (“id”) — Command line name (“baseCommand”) — Input parameters (“inputs”) — Environment requirements (“requirements”) — Docker information (“Docker-Requirement”)

req/app-pck/wf (Requirement 9)

The Application Package CWL Workflow class SHALL contain the following elements: — Identifier (“id”) — Title (“label”) — Abstract (“doc”)

req/app-pck/wf-inputs (Requirement 10)

The Application Package CWL Workflow class “inputs” fields SHALL contain the following elements: — Identifier (“id”) — Title (“label”) — Abstract (“doc”)

req/app-pck/metadata (Requirement 11)

The Application Package CWL Workclass classes SHALL include additional metadata as defined in Table 1

Required in table 1: version

CWL STAC / IO

Inputs and outputs are staged in and out using STAC catalogs. Each input that is an “EO Product” like a vector or raster dataset is declared as type `Directory` and is described by a STAC catalog `catalog.json` and accompanying STAC item(s) placed in that directory.

Outputs are handled in much the same way, the application in the application package is reposible for generating a STAC catalog (also named `catalog.json`) and stage out all the contents of its working directory together with the STAC metadata describing the outputs.

Inputs

There are inputs of type directory in the CommandLineTool description.

```
extract-directory-inputs-from.sh CommandLineTool "$CWL_PATH" || FAIL_REQ_12=1
```

```
Running
inputs_with_directory_type:
- label: hcspots
  doc: hcspots
  type: Directory
  default:
    class: Directory
    location: null
  inputBinding:
    prefix: --hcspots
- label: material_labels
  doc: material_labels
  type: Directory
  default:
    class: Directory
    location: null
  inputBinding:
    prefix: --material-labels
```

```
- label: urban_atlas
  doc: urban_atlas
  type: Directory
  default:
    class: Directory
    location: null
  inputBinding:
    prefix: --urban-atlas
```

There are inputs of type directory in the Workflow description.

```
extract-directory-inputs-from.sh Workflow "$CWL_PATH" || FAIL_REQ_13=1
```

```
Running
inputs_with_directory_type:
- label: hcspots
  doc: hcspots
  type: Directory
  default:
    class: Directory
    location: null
- label: material_labels
  doc: material_labels
  type: Directory
  default:
    class: Directory
    location: null
- label: urban_atlas
  doc: urban_atlas
  type: Directory
  default:
    class: Directory
    location: null
```

Outputs

The CommandLineTool stages out all the contents of the working directory.

```
extract-commandlinetool-outputs.sh "$CWL_PATH" || FAIL_REQ_14=1
```

```
results:
  type: Directory
```

```
outputBinding:
  glob: .
Extracts all items from Working directory: true
```

Requirements validated

req/app-pck-stage-in/clt-stac (Requirement 12)

All input parameters of the CWL ComandLineTool that require the staging of EO products SHALL be of type Directory.

req/app-pck-stage-in/wf-stac (Requirement 13)

Input parameters of the CWL Workflow that require the staging of EO products SHALL be of type Directory.

req/app-pck-stage-out/output-stac (Requirement 14)

The outputs field of the CommandLineTool that requires the stage-out of EO products SHALL retrieve all the files produced in the working directory.

Summary

```
for i in {1..14}; do
  var="FAIL_REQ_$i"
  req=${var#FAIL_}
  if [[ "${!var:-}" == "1" ]]; then
    #echo "$req: FAILED"
    printf '%s\t\033[31mFAILED\033[0m\n' "$req"
  else
    printf '%s\t\033[32mPASSED\033[0m\n' "$req"
  fi
done
```

```
REQ_1  PASSED
REQ_2  PASSED
REQ_3  PASSED
REQ_4  PASSED
REQ_5  PASSED
REQ_6  PASSED
REQ_7  PASSED
REQ_8  PASSED
```

REQ_9	PASSED
REQ_10	PASSED
REQ_11	PASSED
REQ_12	PASSED
REQ_13	PASSED
REQ_14	PASSED